



CS4051NI/CC4059NI Fundamentals of Computing

**70% Individual Coursework
Final Submission**

2024/25 Spring

Student Name: Shyagi Kunwar

London Met ID: 24046505

College ID: NP01NT4A240176

Assignment Due Date: Wednesday, May 14, 2025

Assignment Submission Date: Wednesday, May 14, 2025

Word Count: 3234

Project File Links:

Onedrive Drive Link:	Keep Google Drive URL of your Project Here with Anyone in Organization can View Option Enabled
-----------------------------	--

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Match Groups

-  **11 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **6 Missing Citation 3%**
Matches that have quotation marks, but no in-text citation
-  **1 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 7%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
CTI Education Group on 2017-02-15		<1%
2	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-12-30		<1%
3	Submitted works	
Lincoln University on 2023-12-11		<1%
4	Submitted works	
Wayne State University on 2024-11-23		<1%

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **11 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **6 Missing Citation 3%**
Matches that have quotation marks, but no in-text citation
-  **1 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Table of Contents

1.	Introduction	1
1.1	Aims and Objectives	1
1.2	Technologies and Tools.....	2
2.	Discussion and Analysis	3
2.1	Algorithm.....	3
2.2	Flowchart.....	5
2.3	Pseudocode	5
2.4	Data Structures.....	8
3.	Program	10
4.	Testing	12
5.	Conclusion.....	17
6.	Appendix.....	18

List of Figures

Figure 1: Python Logo	2
Figure 2: IDLE Logo	2
Figure 3: Flowchart.....	5
Figure 4: Invalid Program ID	12
Figure 5: Valid Program ID.....	12
Figure 6: Invalid Input.....	13
Figure 7: Valid Input	13
Figure 8: Purchasing Products	14
Figure 9: Selling Multiple Products Invoice.....	15
Figure 10: Update after sale	16

List of Tables

Table 1: Test 1 12

Table 2: Test 2 13

Table 3: Test 3 14

Table 4: Test 4 14

Table 5: Test 5 15

1. Introduction

This coursework is assigned to us students for the reason of skill and knowledge development on python programming language using IDLE python. This coursework is designed by the teachers to introduce python between the students studying related subjects. Python is one of the most used programming languages throughout the world and it is important that a studying an IT related should know about the basics of python programming. Through this given coursework, students can develop a strong base regarding programming and has also introduced the principle and concepts of python programming language, which is the principle of OOP, attributes, bubble sort and other. This will also help the students to build a strong foundation, and they will be able to face more complex python programming.

The main aim of this coursework is to introduce the new concept of python between the new beginner students studying in first year IT, so the student can learn how to write at least basics of python programming language. At the end of this project students will have a strong build, knowledge and skill about python coding within IDLE python. IDLE is a very good platform for learning the basics and bases of python programming language.

1.1 Aims and Objectives

Aims: -

One of the aims of this coursework is to familiarize the students with the python programming language environment through IDLE. In today's world coding plays a very vital role because every application needs a specific coding, without coding the online world seems a little impossible to imagine. The focus of this coursework is to encourage best coding practises within the students at the college.

Objectives: -

- To introduce the students to python programming language and its importance, using IDLE.
- To help the students understand and apply the concept of python more clearly.
- To provide the students a first-hand experience on building python-based applications.
- To enhance the students problem-solving skills.

1.2 Technologies and Tools

- Python Programming language - Python is a core programming language for developing applications in today's world because it is easy to read, write and understanding.



Figure 1: Python Logo

- IDLE (Integrated Development and Learning Environment) – IDLE is a python build-in application which is simpler to understanding and user-friendly for beginners learning the basics of python.



Figure 2: IDLE Logo

- Text File Storage
- File Handling
- Data Structures
- Modular Programming

2. Discussion and Analysis

2.1 Algorithm

BerryB Wholesale:

1. Start
2. Open “product.txt” file
 - Open the file containing all the product information.
3. Read each line and store in a dictionary.
 - Store with the product ID as the key and product details as the value of the product.
4. Display menu choices
 - Display choices for the user to select:
 - 1 – to Sell
 - 2 – to Restock
 - 3 – to Exit
5. User Input
 - Read the user’s choice.
6. If the user chooses 1 (Sell):
 - Display the products available.
 - Input the Customer’s name and Phone number.
 - Input the product ID and product quantity to sell.
 - Check if product ID is valid:
 - If valid, proceed.
 - If invalid, type the product ID again.
 - Check if quantity is valid:
 - If valid, proceed

- If invalid, type the quantity again.
 - Calculate the free item.
 - Update the stock:
 - Subtract the quantity sold from the stock.
 - Show Updated stock.
 - Ask if another item needs to be sold:
 - If yes, go back to the product display.
 - If no, generate invoice.
7. Generate invoice
- Create an invoice for the item sold.
8. Save the Invoice to file?
- If yes, write the invoice to a file.
 - If no, discard the invoice.
9. If the user chooses 2 (Restock):
- Update the stock for the products.
 - Show the updated stock.
10. If the user chooses 3 (Exit):
- End the program.
11. End

2.2 Flowchart

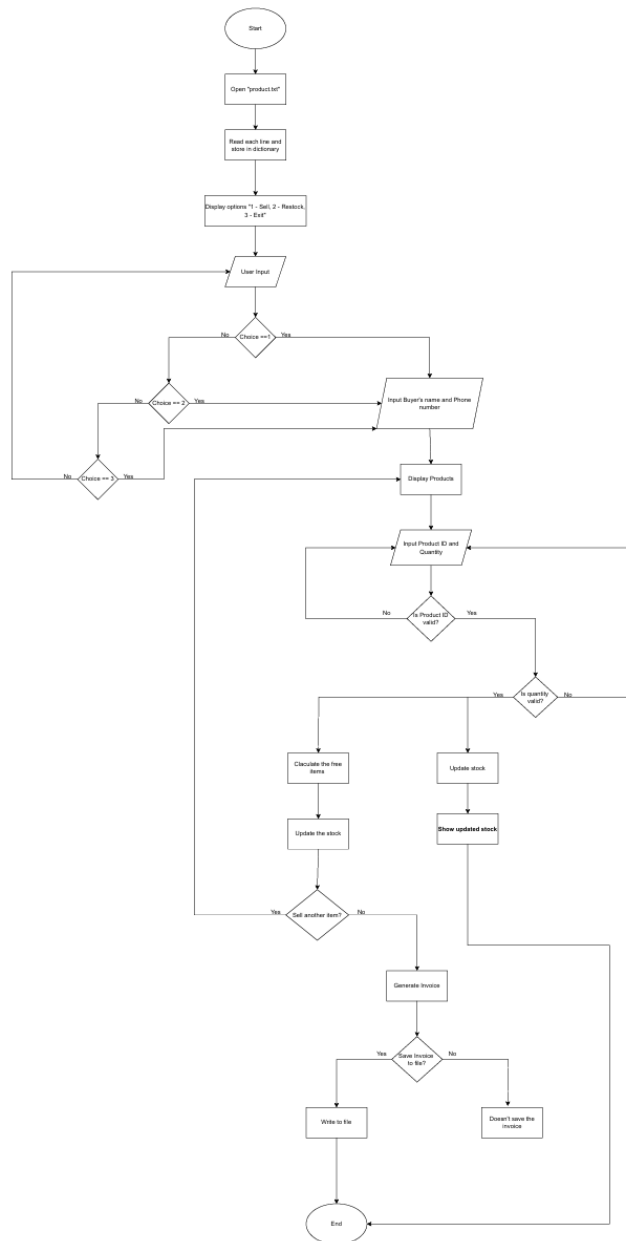


Figure 3: Flowchart

2.3 Pseudocode

BEGIN

Load product data from file and store in dictionary
 CALL read_products ("product.txt") – Store data in 'products'

Display welcome header

Print "Welcome to WeCare Wholesale"

Show menu options repeatedly until the user exits

WHILE user has not selected exit

 Print "1. Sell Product"

 Print "2. Purchase Product"

 Print "3. Exit"

GET user input for choice selection

IF user selects "1. Sell Product" THEN

 CALL sell_product (products)

ELSE IF user selects "2. Purchase Product" THEN

 CALL purchase_item (products)

ELSE IF user selects "3. Exit" THEN

 Print "Thank you. Program ended."

 END program

ELSE

 Print "Invalid option. Try again."

END IF

END WHILE

END

Function to process selling of products

FUNCTION sell_product (data)

 # Get customer information

 GET buyer_name

 GET phone_number

 IF buyer_name is NOT letters OR phone_number is NOT numbers THEN

 Print "Invalid details"

 RETURN

 END IF

 INITIALIZE empty list 'sale_items'

 INITIALIZE 'total_price' as 0

 DO

 CALL show_products (data)

 # Display available products

 GET product ID

 GET quantity user wants to purchase

 IF product ID is invalid OR quantity is invalid THEN

```
        Print "Not Valid"
        CONTINUE
    END IF

    # Get stock details
    GET available_stock FROM data [products_ID]

    # Calculate free items based on "Buy 3, Get 1 Free"
    CALCULATE free_items = quantity
    CALCULATE total_items_needed = quantity + free_items

    IF total_items_needed <= available_stock THEN
        UPDATE stock (deduct purchased quantity)
        ADD product details to sale_items
        Print "You get", free_items, "free units "
    ELSE
        Print "Insufficient stock"
    END IF

    ASK user if they want to sell another product
    WHILE user selects "Yes"

    # Generate and display invoice
    CALL generate_invoice (sale_items, total_price)

    ASK user if they want to save invoice
    IF user selects "Yes" THEN
        CALL save_invoice ("invoice.txt", buyer_name, phone_number, sale_items,
            total_price, current_date)
        Print "Invoice Saved"
    END IF

    CALL update_product_file ("product.txt", data)
    # Save updated stock
    CALL show_products (data)
    # Display new stock levels
ELSE
    Print "Invalid. Try again."
END IF
END FUNCTION

# Function to display available products
FUNCTION show_products (data)
    Print table headers
    Print separator lines
```

```
    FOR EACH product_id IN data
        EXTRACT details from dictionary
        FORMAT data properly to align columns
        Print formatted product details
    END FOR
END FUNCTION

#Function to create and display invoice
FUNCTION generate_invoice (sale_items, total_price)

    FOR EACH item IN sale_items
        FORMAT details correctly
        PRINT structured product details
    END FOR
END FUNCTION

# Function to save updated stock details to file
FUNCTION update_product_file (file_name, data)
    OPEN file in write mode
    FOR EACH product_id IN data
        WRITE updated product details to file
    END FOR
    CLOSE file
END FUNCTION

# Function to store invoice details
FUNCTION save_invoice (file_name, buyer_name, phone_number, items, total_price,
date_str)
    OPEN file 'file_name' in write mode
    WRITE invoice header, customer details, itemized purchases

    FOR EACH item IN items
        FORMAT data correctly and WRITE to file
    END FOR

    WRITE final total price and footer lines
    CLOSE file
END FUNCTION
```

2.4 Data Structures

1. String (str) – String is used in python to store data such as names, addresses, emails, or IDs. Example: - Name = "Shyagi", Address = "Kathmandu", or ID = "NP0231"

Used in code:

```
name = input("Customer Name: ")
```

2. Lists (list) – Lists are the collection that can store multiple values including strings or any numbers. Example: - Names = {"Wes", "Cesar", "Jhonny", "Harry"}, Age = {22, 24, 21, 25}

Used in code:

```
lines = file.readlines()
```

3. Tuples (tuple) – Tuple are somewhat like list the difference is they are immutable which means that they cannot be changed once created. They are used when the data needs to be constant. Example: - Names = {"Wes", "Cesar", "Jhonny", "Harry"}, Age = {22, 24, 21, 25}
4. Dictionaries (dict) – Dictionaries are used to store data in a key-value pair format which means that each data in a dictionary is unique and has a corresponding value making it more manageable. Example: - Members = {"1": {"Name": "Wes Bennet", "Age": "20", "ID": "NT8326"}}
5. Sets (set) – Set is used when there is a need to store collections of unique values. Each value in a set must be unique and the duplicate values are automatically removed. Example: - Modules = {"Fundamentals of Computing", "Programming", "Fundamentals of Cybersecurity", "Introduction to Networking"}
6. DefaultDict – DefaultDict is considered a type of dictionary that provides a default value to a non-existing key which means if you try to access a key that does not exist it'll give the default key on its own.
7. OrderedDict – OrderedDict is a dictionary subclass that always remembers in which order the keys were added. This is very useful when you need to remember the order of the keys like printing items.
8. Counter – Counter is also a subclass of dictionary just like OrderedDict, it counts the frequency of items in a list. This can come very handy such as in case of counting the products sold.
9. Array – Array can be like a list but one main key difference is that it only stores elements of only one data type and it is considered more memory beneficial for larger numeric data.

10. Linked List – Linked List is a linear data structure where the element called node consists of the data and a reference. Linked list does not store items in contiguous memory unlike python normal lists ([]).
11. Stack – Stack is a data structure that runs with the LIFO (Last-In-First-Out) rule. It can be useful in backtracking algorithms.

3. Program

Implementation of the Program

This coursework about WeCare Wholesale is created using Python IDLE. We can perform the following using the program developed:

- Sell Products
- Purchase Products
- We can view the product list
- Generate and save invoices

Sale and Purchase of the product

a. Sell:

- Admin selects "Sell Products"
- Enter customer details
- After providing valid product ID and quantity
- Calculated free items (Buy 3, get 1 free)
- Stock is updated (reduced)
- Invoice is generated and can be saved as per choice

b. Purchase

- Admin selects "Purchase Products"
- Then enters a valid product ID and product quantity
- After the sales the stock is updated
- Stock is also updated in product.txt

Creation of a Text File

- When the sales are successful, an invoice is generated, and it can be saved as well
- The invoice includes customer details, product breakdown and total amount (bill)

Opening Text and Showing the Bill

- The program generates a formatted bill which includes the all the detail
- The saved text file also consists of the same information

Termination

- Admin selects "Exit"
- Program displays "Thank you. Program ended."

4. Testing

Test 1 – Invalid Product ID

Objective	Handle Invalid Product ID
Action	Enter a letter where product ID
Expected Result	Program should show error and print “Invalid Input” or “Please use numbers only”
Actual Result	Program was displaying error correctly and continued running
Conclusion	Program worked as expected

Table 1: Test 1

```

*****
ID  Product Name      Brand Name      Qty   Price   Country
*****
1   Vitamin C Serum   Garnier        200   2000    France
2   Skin Cleanser     Cetaphil       100   560     Switzerland
3   Sunscreen         Aqualogica    200   1400    India
4   Moisturizing Gel  Neutriderm    150   1700    Australia
5   Daily Face Wash   PureEase      160   780     UK
6   Sunblock SPF 50   SunArmor      170   1780    Brazil
7   Anti-Aging Serum  YouthSpring   69    2900    Italy
8   Rose Water Toner  BloomSkin     180   860     Bulgaria
Enter product ID to sell: s
Invalid input.
.....

```

Figure 4: Invalid Program ID

```

*****
ID  Product Name      Brand Name      Qty   Price   Country
*****
1   Vitamin C Serum   Garnier        200   2000    France
2   Skin Cleanser     Cetaphil       100   560     Switzerland
3   Sunscreen         Aqualogica    200   1400    India
4   Moisturizing Gel  Neutriderm    150   1700    Australia
5   Daily Face Wash   PureEase      160   780     UK
6   Sunblock SPF 50   SunArmor      170   1780    Brazil
7   Anti-Aging Serum  YouthSpring   69    2900    Italy
8   Rose Water Toner  BloomSkin     180   860     Bulgaria
Enter product ID to sell: 4
Enter quantity: 5
You get 1 free units.

```

Figure 5: Valid Program ID

Test 2 - Invalid Input

Objective	Handle Invalid Input
Action	Enter a number where name is asked

Expected Result	Program should show error and print “Invalid Input” or “Please use letters only”
Actual Result	Program was displaying error correctly and continued running
Conclusion	Program worked as expected

Table 2: Test 2

```

Menu:
1. Sell Product
2. Purchase Product
3. Exit
-----
Choose option: 1
Customer Name: 3
Invalid name. Please use only letters.
Customer Name: |

```

Figure 6: Invalid Input

```

Menu:
1. Sell Product
2. Purchase Product
3. Exit
-----
Choose option: 1
Customer Name: Wes
Phone Number : |

```

Figure 7: Valid Input

Test 3 – Purchasing Product

Objective	Purchasing Product
-----------	--------------------

Action	Buying products
Expected Result	Program should give output and update the stock
Actual Result	Program gave updated stock and purchase was successful
Conclusion	Program worked as expected

Table 3: Test 3

Choose option: 2

```

*****
ID  Product Name      Brand Name      Qty   Price   Country
*****
1   Vitamin C Serum   Garnier        200   2000    France
2   Skin Cleanser     Cetaphil       100   560     Switzerland
3   Sunscreen         Aqualogica    200   1400    India
4   Moisturizing Gel  Neutriderm    150   1700    Australia
5   Daily Face Wash   PureEase      160   780     UK
6   Sunblock SPF 50   SunArmor      170   1780    Brazil
7   Anti-Aging Serum  YouthSpring    69    2900    Italy
8   Rose Water Toner  BloomSkin     180   860     Bulgaria
Enter product ID to purchase: 2
Enter quantity to add: 40
Stock updated.

```

Figure 8: Purchasing Products

Test 4 – Selling Multiple Product

Objective	Selling Multiple Product
Action	Sell multiple products
Expected Result	Program should show the bill of selling multiple products
Actual Result	Program generated the invoice and product information was displayed correctly
Conclusion	Program worked as expected

Table 4: Test 4

```

-----
BILL - WeCare Wholesale
Customer: Wes
Phone   : 3478356
Date    : 2025-05-14 04:51:08
-----

```

```

-----
Product          Qty  Free  Price  Total
-----
Moisturizing Gel  40    13   1700   68000
Anti-Aging Serum  10     3   2900   29000
-----
Total Amount: 97000
-----

```

Figure 9: Selling Multiple Products Invoice

Test 5 – Product Stock Update

Objective	Check Product Stock Update
Action	Sell or purchase products, then open the product.txt for update
Expected Result	The quantity in product.txt should decrease after sale or increase after purchase
Actual Result	The quantity was updated correctly, in product.txt it was decrease after sale or increase after purchase
Conclusion	Program worked as expected

Table 5: Test 5

```

Menu:
1. Sell Product
2. Purchase Product
3. Exit
-----
Choose option: 1
Customer Name: Wes
Phone Number : 743655238
*****
ID  Product Name      Brand Name      Qty  Price  Country
*****
1   Vitamin C Serum   Garnier         200   2000   France
2   Skin Cleanser     Cetaphil        100   560    Switzerland
3   Sunscreen         Aqualogica     200   1400   India
4   Moisturizing Gel  Neutriderm     150   1700   Australia
5   Daily Face Wash   PureEase       160   780    UK
6   Sunblock SPF 50   SunArmor       170   1780   Brazil
7   Anti-Aging Serum  YouthSpring    69    2900   Italy
8   Rose Water Toner  BloomSkin      180   860    Bulgaria
Enter product ID to sell: 7
Enter quantity: 20

```

Figure 10: Update after sale

5. Conclusion

In this given coursework by my college, we successfully learned to develop a wholesale management system named WeCare Wholesale using Python (IDLE) and modular programming practices. The system we created allows us to manage skincare products by giving us the opportunities such as Selling, Purchasing and Stock management. Through this given project we as students were able to learn the essentials basics of Python, we learned how to use file handling to read and write product and invoice information, how to handle errors and invalid input data, how to display the data clearly and understandably, etc.

We also tested the program with different inputs to fully make sure it worked properly and gave all the data correctly. At the end the program runs smoothly and covers all the features and functions required, such as invoice creation, stock updates and user-friendly interactions

Overall, this coursework helped enhance my understanding of fundamental programming concepts, improved my programming skills and gave me a practical experience in developing a first-hand Python system in the real-world. This project also helped us very much with what to expect when we are having a real-time experience the practical really gave us an insight into Python system.

6. Appendix

1. Read.py

```
def read_products(file_name):
    data_dict = {}
    file = open(file_name, "r")
    lines = file.readlines()
    number = 1
    for line in lines:
        cleaned = ""
        for char in line:
            if char != "\n":
                cleaned += char
        values = cleaned.split(",")
        data_dict[number] = values
        number += 1
    file.close()
    return data_dict
```

2. write.py

```
def save_invoice(file_name, buyer_name, phone, items, total_price, date_str):
    file = open(file_name, "w")
    file.write("WeCare Wholesale Invoice\n")
    file.write("=" * 60 + "\n")
    file.write("Customer: " + buyer_name + "\n")
    file.write("Phone   : " + phone + "\n")
    file.write("Date    : " + date_str + "\n")
    file.write("-" * 60 + "\n")
    file.write("Product      Qty Free Price  Total\n")
    file.write("-" * 60 + "\n")
    for i in items:
        name = i[0][:17]
        qty = str(i[2])
        free = str(i[3])
        rate = str(i[4])
        sub = str(i[5])
        line = name + " " * (18 - len(name)) + qty + " " * (6 - len(qty)) + free + " " * (6 -
len(free)) + rate + " " * (8 - len(rate)) + sub + "\n"
        file.write(line)
    file.write("-" * 60 + "\n")
    file.write("Total Payable: " + str(total_price) + "\n")
    file.write("=" * 60 + "\n")
```



```
file.close()
```

3. operations.py

```
from datetime import datetime
import write
```

```
def show_products(data):
    print("*" * 75)
    print("ID Product Name      Brand Name      Qty Price Country")
    print("*" * 75)
    for pid in data:
        val = data[pid]
        if len(val) >= 5:
            name = val[0]
            brand = val[1]
            qty = val[2]
            price = int(val[3]) * 2
            origin = val[4]

            # Manually adjusting the width for each column using string concatenation
            row = str(pid) + " " # spacing for ID column
            row += name + " " * (18 - len(name)) # padding for Product Name
            row += brand + " " * (18 - len(brand)) # padding for Brand Name
            row += qty + " " * (8 - len(qty)) # padding for Qty column
            row += str(price) + " " * (10 - len(str(price))) # padding for Price column
            row += origin # Country column, no extra padding
            print(row)

def sell_product(data):
    # Validate name
    while True:
        name = input("Customer Name: ")
        if name.isalpha():
            break
        else:
            print("Invalid name. Please use only letters.")

    # Validate phone number
    while True:
        phone = input("Phone Number : ")
        if phone.isdigit():
            break
        else:
```

```
        print("Invalid phone number. Please use digits only.")

sale_list = []
total = 0
cont = True

while cont:
    show_products(data)
    try:
        pid = int(input("Enter product ID to sell: "))
        if pid not in data:
            print("ID not valid.")
            continue
    except:
        print("Invalid input.")
        continue

    try:
        qty = int(input("Enter quantity: "))
    except:
        print("Enter numbers only.")
        continue

    in_stock = int(data[pid][2])
    free_items = qty // 3
    needed = qty + free_items

    if qty <= 0 or needed > in_stock:
        print("Not enough in stock.")
        continue

    print("You get", free_items, "free units.")

    data[pid][2] = str(in_stock - needed)

    item_name = data[pid][0]
    item_brand = data[pid][1]
    selling_price = int(data[pid][3]) * 2
    item_total = selling_price * qty
    total += item_total

    sale_list.append([item_name, item_brand, qty, free_items, selling_price,
item_total])

    again = input("Sell another? (yes/no): ")
    if again.lower() != "yes":
```

```

    cont = False

    now = datetime.now()
    date_info = now.strftime("%Y-%m-%d %H:%M:%S")
    print("=" * 70)
    print("BILL - WeCare Wholesale")
    print("Customer:", name)
    print("Phone :", phone)
    print("Date  :", date_info)
    print("-" * 70)
    print("Product      Qty Free Price  Total")
    print("-" * 70)
    for i in sale_list:
        pname = i[0][:17]
        q = str(i[2])
        f = str(i[3])
        p = str(i[4])
        t = str(i[5])

        # Manually adjusting the width for each column using string concatenation
        line = pname + " " * (18 - len(pname)) + q + " " * (6 - len(q)) + f + " " * (6 - len(f)) + p
        + " " * (8 - len(p)) + t
        print(line)
    print("-" * 70)
    print("Total Amount:", total)
    print("=" * 70)

    ask = input("Do you want to save the bill? (yes/no): ")
    if ask.lower() == "yes":
        filename = "invoice_" + name + "_" + now.strftime("%Y%m%d%H%M%S") + ".txt"
        write.save_invoice(filename, name, phone, sale_list, total, date_info)
        print("Invoice stored as", filename)

    data[pid][2] = str(in_stock - needed)
    print("Stock updated.\n")
    show_products(data)

def purchase_item(data):
    show_products(data)
    try:
        pid = int(input("Enter product ID to purchase: "))
        if pid not in data:
            print("Invalid ID.")
            return
    except:

```

```
    print("Please enter a number.")
    return

try:
    add_qty = int(input("Enter quantity to add: "))
    if add_qty <= 0:
        print("Must be positive.")
        return
except:
    print("Not a valid quantity.")
    return

data[pid][2] = str(int(data[pid][2]) + add_qty)
print("Stock updated.\n")
show_products(data)
```

4. main.py

```
import read
import operations

print("\n")

title = "WeCare Wholesale"
contact = "Hattiban, Lalitpur | Contact: 9835245637"

# Manual centering with padding
width = 60
left_pad_title = " " * ((width - len(title)) // 2)
left_pad_contact = " " * ((width - len(contact)) // 2)

print(left_pad_title + title)
print(left_pad_contact + contact)

print("-" * 60)
print("Logged in as: Admin")
print("-" * 60)

products = read.read_products("product.txt")

while True:
    print("\nMenu:")
    print("1. Sell Product")
    print("2. Purchase Product")
    print("3. Exit")
```

```
print("-" * 30)

try:
    selection = int(input("Choose option: "))
except:
    print("Enter number between 1 and 3.")
    continue

if selection == 1:
    operations.sell_product(products)
elif selection == 2:
    operations.purchase_item(products)
elif selection == 3:
    print("Thank you. Program ended.")
    break
else:
    print("Invalid option. Try again.")
```